

EduPlan Studio

Technical Architecture Specification & Technical Stack Blueprint

TARGET ENGINE
Go 1.22+ / React 18

PRIMARY DB
PostgreSQL 16 + pgvector

ORCHESTRATION
Temporal / Asynq

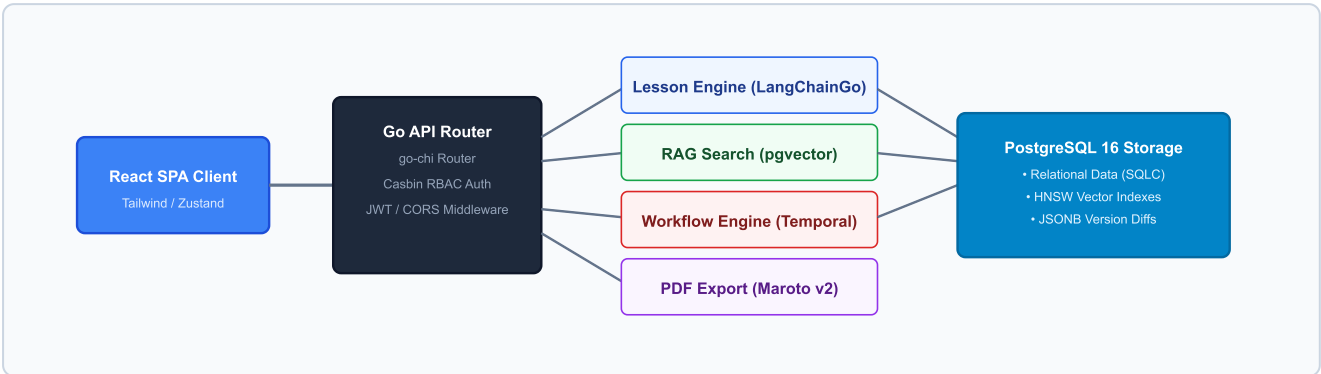
VERSION
v2.4 (Enterprise)

1. Executive Summary & Core Platform Vision

EduPlan Studio is an enterprise edtech backend and administrative control platform designed to unify curriculum resource discovery, AI-assisted lesson plan generation, strict approval workflows, and immutable version control. Built on a high-throughput Golang core with a responsive React frontend, it empowers educators and administrators to maintain academic standards with zero computational overhead.

Key Architectural Pillar: Decoupled, modular Go microservices utilizing SQLC for high-performance type-safe queries, Casbin for dynamic role-based permissions, and LangChainGo with pgvector for semantic curriculum retrieval.

2. High-Level Architecture Diagram



3. Technology Stack & Tooling Matrix

Domain / Layer	Selected Package / Tool	Technical Rationale
HTTP Framework	go-chi/chi	Zero-allocation router, idiomatic context handling, native Go middleware compliance.
Database Access	SQLC	Compiles raw optimized SQL queries directly into type-safe, boilerplate-free Go code.
AI Orchestration	tmc/langchaingo	Native Go library for LLM prompt chaining, JSON schema validation, and embedding generation.
Access Control	casbin/casbin/v2	Enforces RBAC/ABAC models across multi-tenant educational administrative hierarchies.
Workflow State	temporalio/sdk-go	Durable, stateful workflow execution surviving system restarts for multi-tier approvals.
PDF Generation	johnfercher/maroto/v2	Grid-based declarative Go PDF builder without requiring slow browser headless setups.

4. Database Schema Specification (SQLC Compatible)

The system utilizes PostgreSQL 16 with the `pgvector` extension. Below is the production table definition for structured lesson plans and semantic curriculum embeddings.

```
-- Enable pgvector extension for RAG Search
CREATE EXTENSION IF NOT EXISTS vector;

-- Core Lesson Plans Table
CREATE TABLE lesson_plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title VARCHAR(255) NOT NULL,
  subject VARCHAR(100) NOT NULL,
  grade_level VARCHAR(50) NOT NULL,
  author_id UUID NOT NULL REFERENCES users(id),
  status VARCHAR(50) NOT NULL DEFAULT 'DRAFT', -- DRAFT, REVIEW, APPROVED, PUBLISHED
  current_version VARCHAR(20) NOT NULL DEFAULT 'v1.0',
  content_json JSONB NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Version History Log Table (Immutable Delta Storage)
CREATE TABLE lesson_plan_versions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lesson_plan_id UUID NOT NULL REFERENCES lesson_plans(id) ON DELETE CASCADE,
  version_tag VARCHAR(20) NOT NULL,
  snapshot_json JSONB NOT NULL,
  change_summary TEXT NOT NULL,
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Curriculum Standard Embeddings for Semantic RAG
CREATE TABLE curriculum_resources (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code VARCHAR(100) UNIQUE NOT NULL,
  subject VARCHAR(100) NOT NULL,
  description TEXT NOT NULL,
  embedding vector(1536) -- OpenAI / Nomic Vector Size
);

CREATE INDEX idx_curriculum_vector ON curriculum_resources
USING hnsw (embedding vector_cosine_ops);
```

5. AI Generation & Semantic RAG Search Pipeline

The lesson plan generator uses a hybrid RAG approach: standard curriculum frameworks are contextually retrieved via vector similarity and passed into an LLM prompt to output strict JSON.

1. Context Retrieval (RAG)

Queries PostgreSQL using cosine distance on user prompt vectors to find matching educational benchmarks.

```
SELECT code, description
FROM curriculum_resources
ORDER BY embedding <=> $1
LIMIT 3;
```

2. Structured Extraction

Enforces JSON Schema constraint via LangChainGo to populate lesson sections reliably.

```
type LessonOutput struct {
  Objectives []string `json:"obj"`
  Activities []Activity `json:"act"`
}
```

7. Security & Granular Access Control (Casbin Model)

EduPlan Studio uses Attribute and Role-Based Access Control (RBAC/ABAC) driven by Casbin. This prevents unauthorized access across schools, departments, and user roles.

Casbin RBAC Policy (rbac_model.conf)

```
[request_definition]
r = sub, obj, act

[policy_definition]
p = sub, obj, act

[role_definition]
g = _, _

[policy_effect]
e = some(where (p.eft == allow))

[matchers]
m = g(r.sub, p.sub) && r.obj == p.obj &&
r.act == p.act
```

Sample Policy Definitions

```
# Admin Rules
p, admin, lesson_plans, create
p, admin, lesson_plans, publish
p, admin, users, manage

# Teacher Rules
p, teacher, lesson_plans, create
p, teacher, lesson_plans, read

# Role Assignments
g, s.jenkins@academy.edu, admin
g, m.vance@academy.edu, teacher
```

8. Go Microservices Project Repository Layout

The standard Go monorepo structure segregates domain entities, application services, infrastructure, and web assets.

```
eduplan-studio/
├── cmd/
│   ├── api-server/      # Main REST / GraphQL API server entrypoint
│   ├── worker/          # Asynq / Temporal background queue process
│   └── seed/             # Curriculum database seeder
├── internal/
│   ├── domain/          # Entities: LessonPlan, User, Curriculum, Version
│   ├── auth/            # Casbin policies, JWT validation, middleware
│   ├── curriculum/      # RAG pipeline, pgvector similarity search
│   ├── generator/       # LangChainGo integration & structured prompts
│   ├── workflow/        # Temporal workflows, signals, and activities
│   └── storage/          # SQLC generated code & PostgreSQL adapters
├── db/
│   ├── migrations/      # Goose / golang-migrate SQL scripts
│   └── queries/          # SQLC query source files (*.sql)
├── pkg/
│   └── pdf/             # Maroto v2 layout templates & document exporter
└── web/                # React single page frontend application
```

9. Production Deployment & SLA Strategy

Containerized Infrastructure

- Multi-stage Docker builds yielding <20MB static Go binaries.
- Automated horizontal pod autoscaling (HPA) on Kubernetes.
- Redis cluster backed background queue isolation via Asynq.

Reliability & SLA Targets

- **99.9% Uptime:** Zero-downtime rolling deployments.
- **Sub-100ms API Latency:** Pre-compiled SQLC queries.
- **RAG Response:** <1.2s embedding & extraction loop.